

Builder Design Pattern in C# with Examples

In this article, I will discuss the **Builder Design Pattern in C#** with Examples. Please read our previous article discussing the [Abstract Factory Design Pattern in C#](#) with Examples. The Builder Design Pattern falls under the category of the Creational Design Pattern. As part of this article, we will discuss the following pointers.

1. [What is the Builder Design Pattern?](#)
2. [Understanding the Builder Design Pattern with Real-Time Example.](#)
3. [Understanding the class diagram of the Builder Design Pattern?](#)
4. [Implementing the Builder Design Pattern in C#: Generating Different Reports and Beverages](#)
5. [When to use the Builder Design Pattern in Real-time Applications.](#)

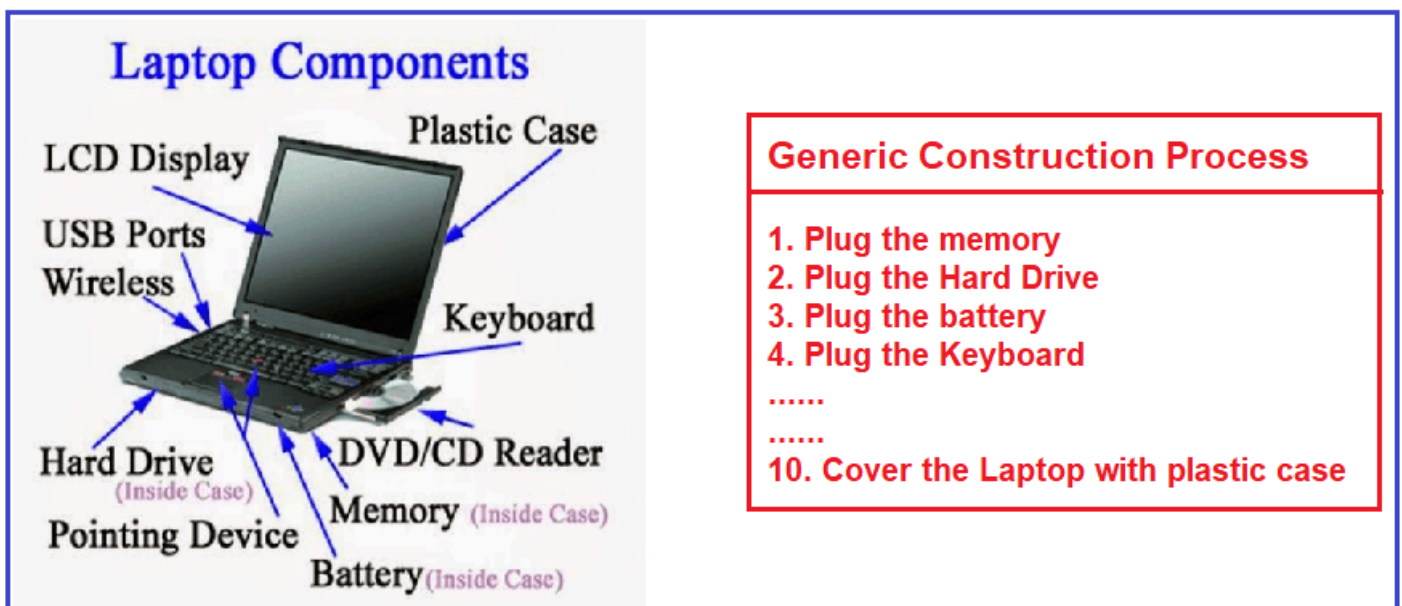
What is the Builder Design Pattern?

According to GOF, the Builder Design Pattern builds a complex object using many simple objects and a step-by-step approach. The Process of constructing the complex object should be so generic that the same construction process can be used to create different representations of the same complex object.

So, the Builder Design Pattern is about separating the construction process of a complex object from its representation, allowing the same construction process to create different representations. When the construction process of the object is very complex, we only need to use the Builder Design Pattern.

Real-Time Example to Understand Builder Design Pattern

Please have a look at the following diagram. Here, the Laptop is a complex object. To build a laptop, we need to use many small objects like LCD Displays, USB Ports, Wireless, Hard Drives, Pointing Devices, Battery, Memory, DVD/CD Reader, Keyboard, Plastic Case, etc. So, we have to assemble all these small objects to build the laptop complex objects.



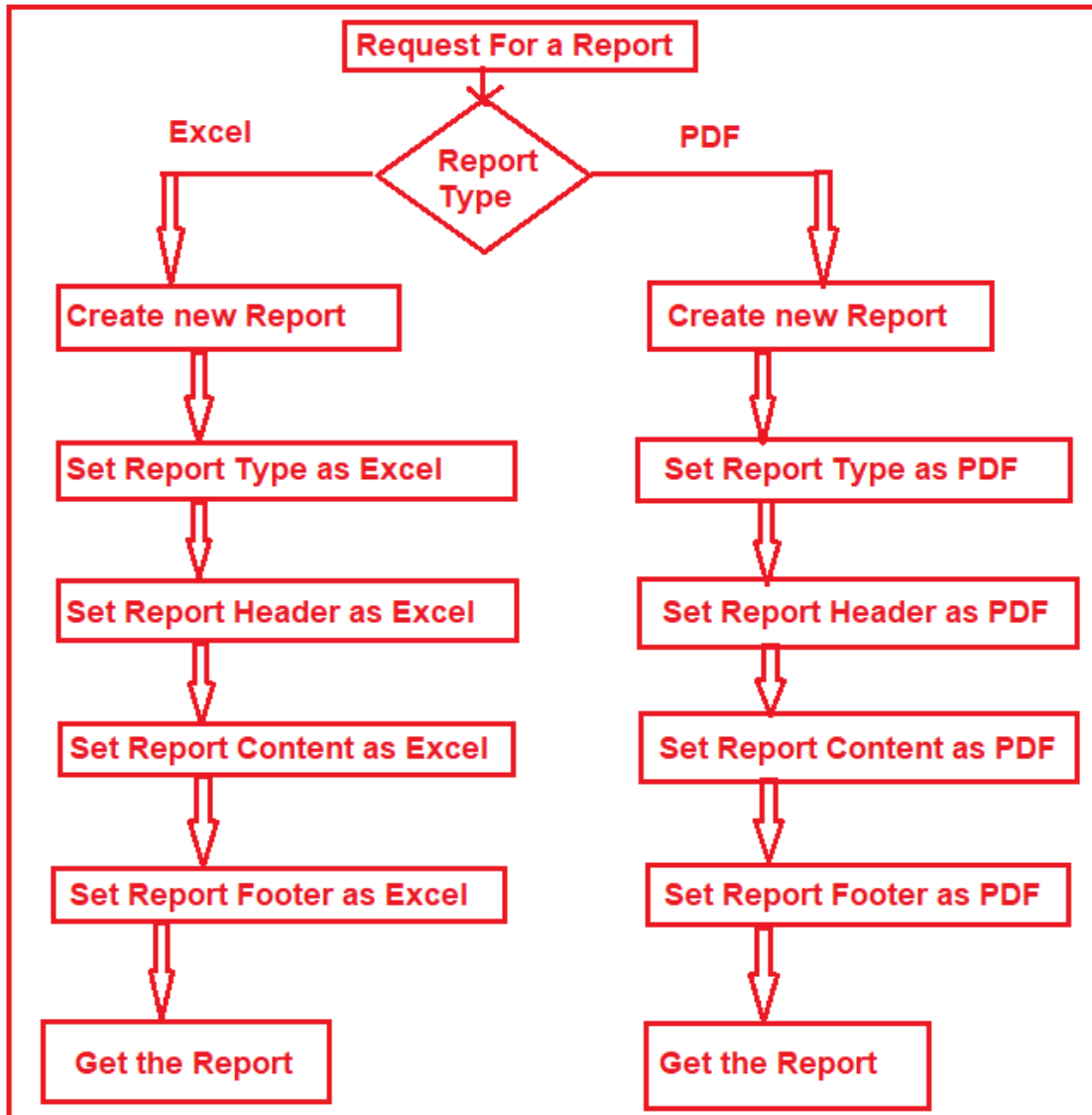
So, to build the complex object laptop, we need to define some generic process, something like below.

1. Plug the memory
2. Plug the Hard Drive
3. Plug the battery
4. Plug the Keyboard
-
-
10. Cover the Laptop with a plastic case

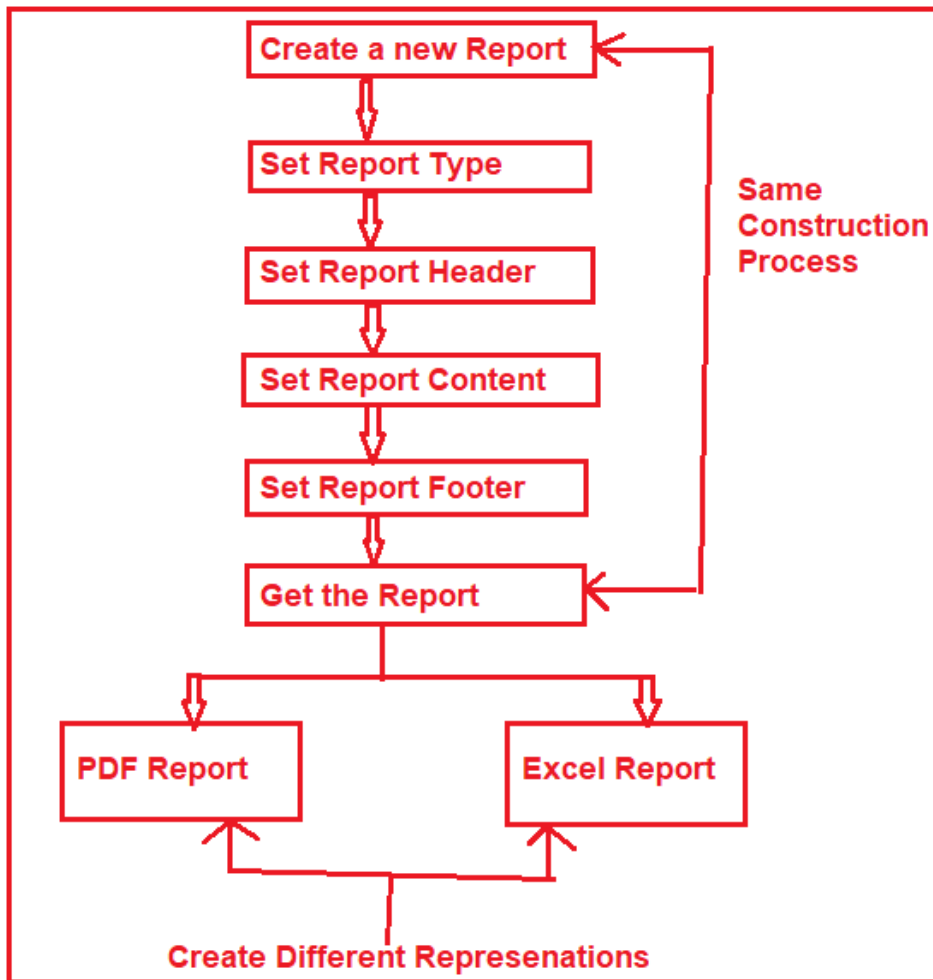
Using the above generic process, we can create different types of laptops, such as those with 14-inch or 17-inch screens and 4GB RAM or 8GB RAM. All laptop creations will follow the same generic process. So, now, if you read the definition, you will understand the definition of the Builder Design Pattern.

Example to Understand Builder Design Pattern:

Let us understand the Builder Design Pattern in C# with one Real-Time Example. Suppose we want to develop an application to display reports. We need to display the reports in Excel or PDF format, meaning we have two types of report representation. To understand this better, please have a look at the following diagram.

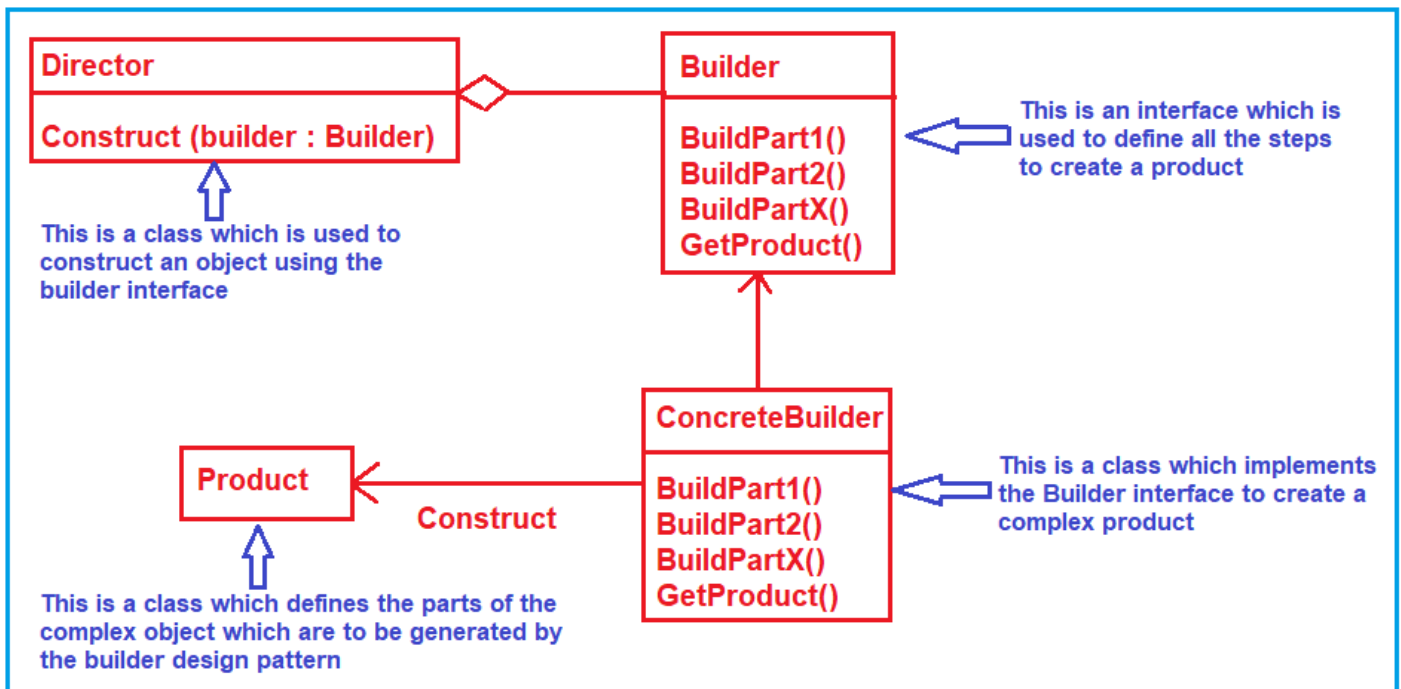


As you can see in the above image, we generate the report in Excel or PDF formats. The construction process involves several steps, such as Creating a new report and setting the report type, header, content, and footer. If you look at the final output, we have one PDF representation and one Excel representation. Please look at the following diagram to understand the construction process and its representation. As you can see, the construction process is the same, but we are getting two types of reports using the same construction process.



Understanding the UML (or Class) Diagram of Builder Design Pattern in C#

Let us understand the UML or Class Diagram and the different components of the Builder Design Pattern. To understand this, please have a look at the following diagram.



The builder design pattern involves four components to separate the construction process from its representation. They are as follows:

1. **Abstract Builder:** The Builder is an interface defining all the steps to make the concrete product.
2. **Concrete Builder:** The Concrete Builder Classes implement the Abstract Builder interface and provide implementation to all the abstract methods. By implementing the builder interface, the concrete builder is responsible for constructing and assembling the individual parts of the product.
3. **Director:** The Director takes those individual processes from the Builder and defines the sequence to build the product.
4. **Product:** The Product is a class, and we want to create this product object using the builder design pattern. This class defines different parts that will make the product.

Note: If you don't understand the above component now, don't worry. Once we see the examples, we will compare them with the above UML diagram. I am sure that after that, you will understand the above UML Diagram and the different components of the Builder Design Pattern.

Implementation of Builder Design Pattern in C#:

Let us implement the Excel and PDF Report example we discussed using the Builder Design Pattern in C# step by step.

Step 1: Creating the Product

Create a class file named **Report.cs** and copy and paste the following code. This is our product class, and within it, we define the attributes (such as ReportHeader, ReportType, ReportFooter, and ReportContent) that are common to create a report. We also define one method, i.e., DisplayReport, to display the report details in the console.

```
using System;
namespace BuilderDesignPattern
{
    // It makes sense to use the Builder Design Pattern only
    // when your products are quite complex
    // and require extensive configuration.
    // Using the following Report Product class, we can configure different
    types of Product
    public class Report
    {
        public string ReportType { get; set; }
        public string ReportHeader { get; set; }
        public string ReportFooter { get; set; }
        public string ReportContent { get; set; }
        public void DisplayReport()
        {
            Console.WriteLine("Report Type :" + ReportType);
            Console.WriteLine("Header :" + ReportHeader);
            Console.WriteLine("Content :" + ReportContent);
            Console.WriteLine("Footer :" + ReportFooter);
        }
    }
}
```

Once we know the definition of the Product we are building, we need to create the Builder.

Step 2: Creating the Abstract Builder Class.

Create a class file named **ReportBuilder.cs** and then copy and paste the following. This abstract class will provide the blueprint to create different types of products. As it is abstract, it can have the capabilities to have both abstract and non-abstract methods. The subclasses will implement this ReportBuilder abstract class and need to provide implementations for all the abstract methods it contains. The Builder Abstract Class specifies methods for creating the different parts of the Product objects.

```
namespace BuilderDesignPattern
{
    // The Builder Abstract Class specifies methods for creating the
    // different parts
    // of the Product objects.
    public abstract class ReportBuilder
    {
        protected Report reportObject;
        public abstract void SetReportType();
        public abstract void SetReportHeader();
        public abstract void SetReportContent();
        public abstract void SetReportFooter();
        public void CreateNewReport()
        {
            reportObject = new Report();
        }
        public Report GetReport()
        {
            return reportObject;
        }
    }
}
```

Notice that we have four abstract methods. Each subclass of ReportBuilder will need to implement those four abstract methods to build a report properly. By implementing the above ReportBuilder interface, we need to create a few concrete builder classes.

Step 3: Creating Concrete Builder Classes.

The Concrete Builder classes implement the Builder interface and provide specific implementations of the building steps. The Application may have several variations of Builders implemented differently. In our example, we are dealing with two types of reports, i.e., Excel and PDF. So, we need to create two concrete builder classes by implementing the ReportBuilder Abstract Class and providing implementation to the ReportBuilder Abstract Methods.

ExcelReport.cs

Create a class file named **ExcelReport.cs** and copy and paste the following code. This ExcelReport class implements the ReportBuilder abstract class and the four abstract methods, the blueprint for creating the report objects. This class is used to provide the blueprint for creating the Excel Report.

```
namespace BuilderDesignPattern
{
    // The Following Concrete Builder Implementd the ReportBuilder Abstract
    // Class and
    // provide specific implementations of the steps for Creating
    // ExcelReport.
    public class ExcelReport : ReportBuilder
    {
        public override void SetReportContent()
```

```

    {
        reportObject.ReportContent = "Excel Content Section";
    }
    public override void SetReportFooter()
    {
        reportObject.ReportFooter = "Excel Footer";
    }
    public override void SetReportHeader()
    {
        reportObject.ReportHeader = "Excel Header";
    }
    public override void SetReportType()
    {
        reportObject.ReportType = "Excel";
    }
}
}

```

PDFReport.cs

Create a class file named **PDFReport.cs** and copy and paste the following code. This class also implements the ReportBuilder abstract class and the four abstract methods, the blueprint for creating the report objects. The following PDFReport class is used to create the Report in PDF format.

```

namespace BuilderDesignPattern
{
    // The Following Concrete Builder Implementd the ReportBuilder Abstract
    Class and
    // provide specific implementations of the steps for Creating
    PDFReport.
    public class PDFReport : ReportBuilder
    {
        public override void SetReportContent()
        {
            reportObject.ReportContent = "PDF Content Section";
        }
        public override void SetReportFooter()
        {
            reportObject.ReportFooter = "PDF Footer";
        }
        public override void SetReportHeader()
        {
            reportObject.ReportHeader = "PDF Header";
        }
        public override void SetReportType()
        {
            reportObject.ReportType = "PDF";
        }
    }
}

```

Once we have the required Concrete Builder Classes, we need to create the Director. The Director will execute the required steps in a particular order to create a particular product.

Step 4: Creating the Director

The Director Class in Builder Design Pattern is only responsible for executing the building steps in a defined order. These steps are so generic that they will produce different products.

So, please create a class file named **ReportDirector.cs** and copy and paste the following code. The following class has one generic method, MakeReport, which will take the ReportBuilder instance as an input parameter and then create and return a particular report object. This class will call the respective methods of the ReportBuilder object (i.e., an object of either ExcelReport or PDFReport) in a particular order to create a particular format report.

```
namespace BuilderDesignPattern
{
    // The Director is only responsible for executing the building steps in
    // a particular order.
    // It is helpful when producing products according to a specific order
    // or configuration.
    public class ReportDirector
    {
        public Report MakeReport(ReportBuilder reportBuilder)
        {
            reportBuilder.CreateNewReport();
            reportBuilder.SetReportType();
            reportBuilder.SetReportHeader();
            reportBuilder.SetReportContent();
            reportBuilder.SetReportFooter();
            return reportBuilder.GetReport();
        }
    }
}
```

Note: This MakeReport Method is so generic that it can create and return different report objects. Once we have the Director and Concrete Builder, the Client can use them to create different types of Reports. In our example, the Client is nothing but the Main method of the Program class.

Step 5: Client Code

The client code in the Builder Design Pattern creates a builder object and passes it to the director, who initiates the construction process. The end result is the product retrieved from the builder object.

In our example, the Main method of the Program class is going to be the Client. So, please modify the Main Method of the Program class as shown below. Here, first, we will create an instance of the ReportDirector class and then create an instance of the PDFReport class. Once we have the ReportDirector instance and PDFReport instance, we call the MakeReport method on the ReportDirector instance, bypassing the PDFReport instance as an argument that will create and return the report in PDF format. The same process is used for Excel reports.

```
using System;
namespace BuilderDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            // Constructing the PDF Report
            // Step1: Create a Builder Object
            // Creating PDFReport Builder Object
            PDFReport pdfReport = new PDFReport();
        }
    }
}
```

```

        // Step2: Pass the Builder Object to the Director
        // First Create an instance of ReportDirector
        ReportDirector reportDirector = new ReportDirector();
        // Then Pass the pdfReport Builder Object to the MakeReport
Method of ReportDirector
        // The ReportDirector will return one of the Representations of
the Product
        Report report = reportDirector.MakeReport(pdfReport);
        // Step3: Display the Report by calling the DisplayReport
method of the Product
        report.DisplayReport();
        Console.WriteLine("-----");
        // Constructing the Excel Report
        // The Process is going to be the same
        ExcelReport excelReport = new ExcelReport();
        report = reportDirector.MakeReport(excelReport);
        report.DisplayReport();
        Console.ReadKey();
    }
}
}

```

Now run the application, and you should get the output shown in the image below.

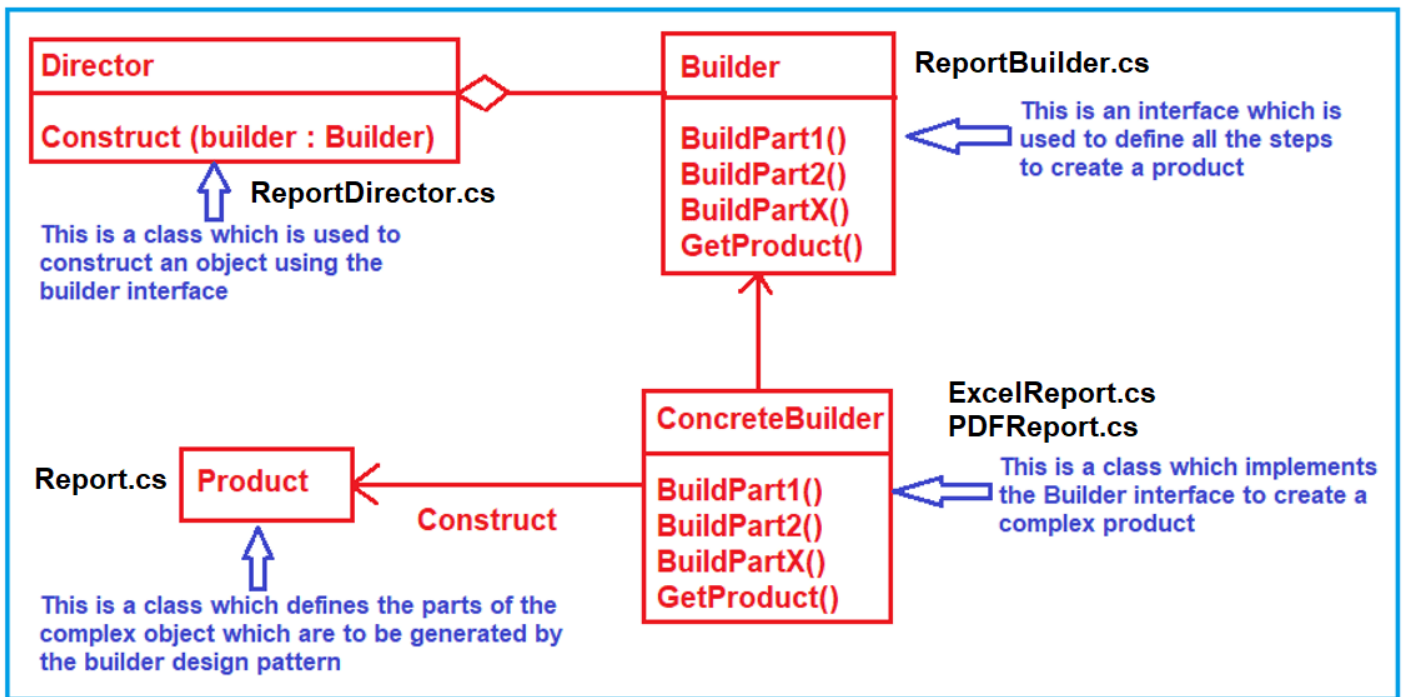
```

Report Type :PDF
Header :PDF Header
Content :PDF Content Section
Footer :PDF Footer
-----
Report Type :Excel
Header :Excel Header
Content :Excel Content Section
Footer :Excel Footer

```

Builder Design Pattern Components in Our Example

Now, let us see the Builder Design Pattern UML Diagram Component with our Example so that you can easily understand the UML Diagram.



Builder Design Pattern Components in Our Example

1. **Client:** Main Method of Program Class
2. **Builder:** ReportBuilder.cs
3. **Concrete Builder:** ExcelReport.cs and PDFReport.cs
4. **Director:** ReportDirector.cs
5. **Product:** Report.cs

Another Real-Time Example Builder Design Pattern – Beverage

Let's say you want to prepare Coffee. To prepare coffee, we need to use Water, Milk, Sugar, and Coffee Products. What we need to do is mix all these components and prepare coffee, which is shown in the image below. So, Coffee (Right-Hand Side) is the complex object, and the small components (Milk, Water, Sugar, and Coffee products) are the simple objects. So, using these simple objects, you can prepare a complex object, i.e., coffee.



Add Water



Add Milk



Add Sugar



Add Coffee Powder



Coffee

Suppose you want to prepare Tea. Then, you need to use Water, Milk, Sugar, and Tea Products. So, using these small objects (Water, Milk, Sugar, and Tea Products), you can prepare a complex object, i.e., Tea, shown in the image below.



Add Water



Add Milk



Add Sugar

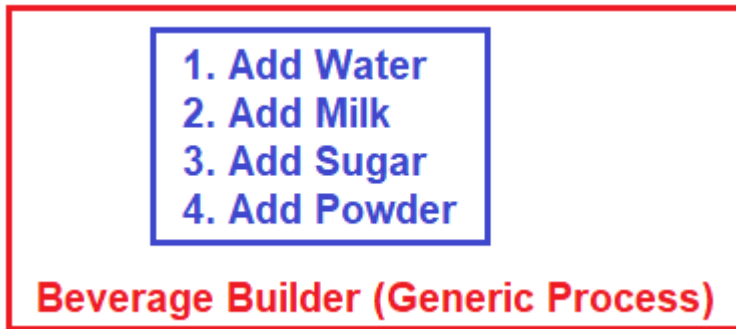


Add Tea Powder



Tea

As we already discussed, the Builder Design Pattern in C# is used to build a complex object using a generic step-by-step process so that the same construction process, i.e., the same steps, can be used to build different representations of the same complex object. If you observe both examples, you will see that the process of preparing tea and coffee is the same, but the content is different. So you can create one generic process, something like the one below.



Using the above generic process, you can now prepare Tea, Coffee, Horlicks, etc. Using the Builder Design Pattern, you can easily implement the above in any programming language. Let us proceed and see how we can implement the above Real-Time Application using the Builder Design Pattern in C#.

Implementation of Builder Design Pattern Real-time Example in C# – Beverage

Let us implement the above real-time example using the Builder Design Pattern in C# step by step.

Step 1: Creating Product (Beverage)

Create a class file named **Beverage.cs** and copy and paste the following code. This will be our product, and in this class, we put all the attributes such as Water, Milk, Sugar, Powder Quantity, and Beverage names that are common to prepare beverages (e.g., Coffee or Tea). We also create one method (e.g., ShowBeverage) that returns the beverage details.

```
namespace BuilderDesignPatternExample
{
    public class Beverage
    {
        public int Water { get; set; }
        public int Milk { get; set; }
        public int Sugar { get; set; }
        public int PowderQuantity { get; set; }
        public string BeverageName { get; set; }

        public string ShowBeverage()
        {
            return "Hot " + BeverageName + " [" + Water + " ml of water, "
+ Milk + "ml of milk, " + Sugar
+ " gm of sugar, " + PowderQuantity + " gm of "
+ BeverageName + "]\n";
        }
    }
}
```

Step 2: Creating Abstract Builder (BeverageBuilder)

The Builder Abstract Class specifies methods for creating the different parts of the Product objects. Create a class file named **BeverageBuilder.cs** and copy and paste the following. This BeverageBuilder class will be abstract and act as a blueprint for any subclasses wanting to create a Beverage. So, it will have different subclass

implementations for different beverage types, such as Coffee, Tea, Horlicks, etc. This class has different abstract methods to set the Milk, Water, Sugar, Powder Quantity, and Beverage Type.

```
namespace BuilderDesignPatternExample
{
    public abstract class BeverageBuilder
    {
        protected Beverage beverage;

        public void CreateBeverage()
        {
            beverage = new Beverage();
        }
        public Beverage GetBeverage()
        {
            return beverage;
        }

        public abstract void SetBeverageType();
        public abstract void SetWater();
        public abstract void SetMilk();
        public abstract void SetSugar();
        public abstract void SetPowderQuantity();
    }
}
```

Step 3: Creating Concrete Builder (CoffeeBuilder and TeaBuilder)

In our example, we will create two types of beverages, i.e., Tea and Coffee. So, we need to create two concrete builder classes by implementing the BeverageBuilder abstract class and providing implementation to all the abstract methods.

CoffeeBuilder.cs

Create a class file named **CoffeeBuilder.cs** and copy and paste the following code. The CoffeeBuilder class creates coffee by mixing the components such as water, milk, sugar, and coffee powder. This CoffeeBuilder class implements the BeverageBuilder abstract class, the blueprint for creating the beverage objects.

```
using System;
namespace BuilderDesignPatternExample
{
    public class CoffeeBuilder : BeverageBuilder
    {
        public override void SetWater()
        {
            Console.WriteLine("Step 1 : Boiling water");
            GetBeverage().Water = 40;
        }
        public override void SetMilk()
        {
            Console.WriteLine("Step 2 : Adding milk");
            GetBeverage().Milk = 50;
        }
        public override void SetSugar()
        {

```

```

        Console.WriteLine("Step 3 : Adding Sugar");
        GetBeverage().Sugar = 10;
    }
    public override void SetPowderQuantity()
    {
        Console.WriteLine("Step 4 : Adding 15 Grams of coffee powder");
        GetBeverage().PowderQuantity = 15;
    }
    public override void SetBeverageType()
    {
        Console.WriteLine("Coffee");
        GetBeverage().BeverageName = "Coffee";
    }
}
}

```

TeaBuilder.cs

Create a class file named **TeaBuilder.cs** and copy and paste the following code. This class also implements the BeverageBuilder abstract class and provides implementation to all the abstract methods. The following TeaBuilder class creates Tea by mixing the components such as Water, Milk, Sugar, and Tea powder.

```

using System;
namespace BuilderDesignPatternExample
{
    public class TeaBuider : BeverageBuilder
    {
        public override void SetWater()
        {
            Console.WriteLine("Step 1 : Boiling water");
            GetBeverage().Water = 50;
        }
        public override void SetMilk()
        {
            Console.WriteLine("Step 2 : Adding milk");
            GetBeverage().Milk = 60;
        }
        public override void SetSugar()
        {
            Console.WriteLine("Step 3 : Adding Sugar");
            GetBeverage().Sugar = 15;
        }
        public override void SetPowderQuantity()
        {
            Console.WriteLine("Step 4 : Adding 20 Grams of coffee powder");
            GetBeverage().PowderQuantity = 20;
        }
        public override void SetBeverageType()
        {
            Console.WriteLine("Tea");
            GetBeverage().BeverageName = "Tea";
        }
    }
}

```

Step 4: Creating the Director (BeverageDirector)

Once you have created the concrete builder classes, you need to create the director. The Director is only responsible for executing the building steps in order, which is helpful when producing products according to a specific order. So, create a class file named **BeverageDirector.cs** and copy and paste the following code. This class has one generic method, which will take BeverageBuilder as an input parameter and then create and return a particular beverage object.

```
namespace BuilderDesignPatternExample
{
    public class BeverageDirector
    {
        public Beverage MakeBeverage(BeverageBuilder beverageBuilder)
        {
            beverageBuilder.CreateBeverage();
            beverageBuilder.SetBeverageType();
            beverageBuilder.SetWater();
            beverageBuilder.SetMilk();
            beverageBuilder.SetSugar();
            beverageBuilder.SetPowderQuantity();

            return beverageBuilder.GetBeverage();
        }
    }
}
```

Note: This MakeBeverage Method is so generic that it can create and return different Beverage objects.

Step 5: Client Code.

Please modify the Main method as shown below. First, we will create an instance of the BeverageDirector class and then an instance of the TeaBuilder class. Once we have the BeverageDirector and TeaBuilder instances, we call the MakeBeverage method on the BeverageDirector instance, bypassing the TeaBuilder instance as an argument that will create and return the tea. Again, we need to follow the same process to make coffee.

```
using System;
namespace BuilderDesignPatternExample
{
    class Program
    {
        static void Main(string[] args)
        {
            Beverage beverage;
            BeverageDirector beverageDirector = new BeverageDirector();

            TeaBuider tea = new TeaBuider();
            beverage = beverageDirector.MakeBeverage(tea);
            Console.WriteLine(beverage.ShowBeverage());
            CoffeeBuilder coffee = new CoffeeBuilder();
            beverage = beverageDirector.MakeBeverage(coffee);
            Console.WriteLine(beverage.ShowBeverage());
            Console.ReadKey();
        }
    }
}
```

Output:

```
Tea
Step 1 : Boiling water
Step 2 : Adding milk
Step 3 : Adding Sugar
Step 4 : Adding 20 Grams of coffee powder
Hot Tea [50 ml of water, 60ml of milk, 15 gm of sugar, 20 gm of Tea]

Coffee
Step 1 : Boiling water
Step 2 : Adding milk
Step 3 : Adding Sugar
Step 4 : Adding 15 Grams of coffee powder
Hot Coffee [40 ml of water, 50ml of milk, 10 gm of sugar, 15 gm of Coffee]
```

When to use the Builder Design Pattern in C#?

The Builder Design Pattern is useful when we need to construct complex objects step by step, allowing the same construction process to create different representations of the object. In C#, the Builder Design Pattern is highly effective in scenarios where constructing an object involves multiple steps. The following are some of the real-time applications where we might consider using the Builder Design Pattern:

- **Complex Constructors:** When an object requires a complex construction process that involves more than just a few steps or requires a significant setup of various fields and nested objects, the Builder Design Pattern can simplify these tasks by spreading the construction across multiple methods within a builder class.
- **Immutable Objects:** When building an immutable object that does not allow modification after its creation, using a Builder makes it possible to set all of its attributes at creation time and then expose the final object without the ability to alter it.
- **Constructing Composite Trees:** In scenarios where you are building a complex tree structure, such as a document with various elements (like paragraphs, images, tables), the Builder Design Pattern provides a mechanism to ensure that different nodes or parts of the tree can be constructed in isolation and then composed as needed.
- **Multiple Representations of an Object:** If your application needs to create different representations of a product (for example, different types of reports that might share the same construction process but have different features or specifications), the Builder Design Pattern allows the construction process to reuse the same building steps while varying the product details.

In the next article, I will discuss the [Builder Design Pattern in Real-time Example in C#](#). Here, in this article, I try to explain the **Builder Design Pattern in C#** with Examples. I hope you understand the need and use of the Builder Design Pattern in C#.